

代号：传说

Godot新手入门与2.5D实战教学

从零开始 | 每步讲原理 | 2.5D动作RPG专供 | Godot 4.6.2

适用对象	Godot零基础，有游戏设计文档
引擎版本	Godot 4.6.2
配套文档	《代号：传说》设计文档 Beta v0.37
教学目标	独立搭建2.5D战斗原型

一、Godot是什么——三个核心概念

在动手操作之前，你必须理解Godot的三个核心概念。这三个概念贯穿Godot的一切操作，理解了它们，后面所有操作你都知道"为什么"。

1.1 节点（Node）——最小的功能砖块

节点是Godot的基本单位，就像乐高积木的每一块。每个节点只做一件事：Sprite2D负责显示图片，AudioStreamPlayer负责播放声音，CharacterBody3D负责3D物理移动。你不需要自己写"显示图片"的代码——Godot已经帮你写好了，你只需要把Sprite2D节点"放进"场景里，然后在右侧属性面板填入图片路径就行了。

这就像装修房子：你不需要自己造门造窗，你去建材市场买一扇门（Door节点）、买一扇窗（Window节点），把它们安装到墙上（场景里），再调节尺寸和位置（属性面板）。Godot提供了上百种现成节点，覆盖了游戏开发中绝大多数需求。

原理：节点本质上是一个GDScript类（C++底层实现）。当你在编辑器中添加一个Sprite2D节点时，Godot在底层创建了一个Sprite2D类的实例，并将它挂载到场景树上。你在属性面板修改的每个值，实际上是在调用这个实例的setter方法。

1.2 场景（Scene）——节点的组合体

场景是节点的组合。一个场景 = 一棵节点树。就像一辆汽车不是单个零件，而是发动机+底盘+轮胎+方向盘的组合体。在Godot中，你创建一个场景，往里面添加各种节点，让它们形成父子关系，这就构成了一个功能完整的"东西"。

比如"玩家"就是一个场景：根节点是CharacterBody3D（负责3D物理移动），子节点有Sprite3D（负责显示角色图片）、CollisionShape3D（负责碰撞检测）、Camera3D（负责摄像机跟随）。这4个节点组合在一起，就是一个能移动、能显示、能碰撞、能被看到的玩家角色。

场景的关键特性是"可复用"——你创建了一个敌人场景，就可以在10个地方放置10个同样的敌人，修改场景文件时所有实例同步更新。这就是为什么Godot的场景系统叫"实例化"（Instance），和面向对象编程中"类与实例"的概念完全一致。

原理：场景文件(.tscn)本质上是一个文本文件，用缩进表示节点的父子层级关系。当你"实例化"一个场景时，Godot读取这个文本文件，递归地创建所有节点并构建父子关系，最终得到一棵完整的节点树。实例化不是"复制"，而是"引用"——修改原始场景会影响所有实例。

1.3 信号（Signal）——节点之间的对话方式

信号是节点之间通信的机制，就像现实中的"广播"——某个节点发出信号（"我受伤了！"），其他节点如果"监听"了这个信号，就会收到通知并做出反应（"播放受伤动画"）。发送信号的节点不需要知道谁在监听，监听的节点也不需要知道信号从哪来，它们完全解耦。

游戏中的例子：按钮被点击时发出pressed信号，Area3D检测到碰撞时发出body_entered信号，Timer计时结束时发出timeout信号。你只需要在编辑器中把信号"连接"到某个函数上，当信号触发时那个函数就会自动执行。

原理：信号是观察者模式的实现。每个信号内部维护一个"回调列表"（Callable数组），调用emit_signal()时，Godot遍历这个列表依次调用每个回调函数。这就是为什么信号是"一对多"的——一个信号可以连接到多个函数，一个函数也可以接收多个信号。

1.4 三个概念的关系

概念	类比	游戏中的例子	一句话理解
节点(Node)	乐高积木	Sprite3D显示图片	一个节点做一件事
场景(Scene)	乐高作品	玩家=移动+显示+碰撞	多个节点组合成功能体
信号(Signal)	广播通知	受伤时通知UI更新血条	节点之间松耦合通信

表1-1 Godot三个核心概念

二、Godot编辑器界面——你的工作台

打开Godot 4.6.2后，你会看到一个分为多个区域的界面。每个区域有明确的用途，理解布局后操作会非常高效。以下按你实际使用的频率从高到低介绍。

面板名称	位置	你在这里做什么	使用频率
视口(Viewport)	正中央	看到场景的样子，拖拽移动节点	每时每刻
场景树(Scene)	左上	添加/删除/排列节点，看节点层级	非常频繁
属性面板(Inspector)	右上	修改节点的属性值（位置/图片/参数）	非常频繁
文件系统(FileSystem)	左下	浏览项目文件，拖拽图片/音频到场景	频繁
输出(Output)	底部	看打印日志和报错信息	调试时
节点信号(Node)	右侧第二个标签	连接信号到函数	中等

表2-1 编辑器面板说明

最常用的操作模式：在场景树中选中节点 -> 在属性面板中修改参数 -> 在视口中看效果。这个循环覆盖了80%的日常工作。信号连接占另外15%——选中节点后切换到"Node"标签页，双击信号名，选择接收函数即可。剩下的5%是写代码。

视图切换快捷键：F2 = 2D视图，F3 = 3D视图，F5 = 运行项目，F6 = 运行当前场景。这三个快捷键你每天会按几百次，建议立刻记住。

三、创建你的第一个Godot项目

步骤1：打开Godot项目管理器

启动Godot后首先看到的是项目管理器（Project Manager）。这是Godot的"大厅"——你所有项目的入口。点击右上角的"新建"(New)按钮，弹出项目创建对话框。

步骤2：填写项目信息

项目名称填"glory_chronicle"（代号传说的英文名，全小写+下划线，这是编程界的命名规范）。项目路径选择你的工作目录，比如"D:/GodotProjects/glory_chronicle"。渲染器选择"Forward+"（Forward+是Godot 4的高质量3D渲染器，我们的2.5D方案需要3D场景，所以必须选3D渲染器）。点击"创建并编辑"(Create and Edit)。

原理：Godot提供三种渲染器——Compatibility（兼容模式，OpenGL3，旧设备用）、Forward+（高质量3D，Vulkan，支持高级光照和阴影）、Forward Mobile（移动端优化版）。2.5D项目需要3D场景+光照，Forward+是唯一正确的选择。选错渲染器后可以在project.godot中修改，但建议一开始就选对。

步骤3：认识空项目界面

项目创建后你会看到一个空白的编辑器界面。左上角的场景树是空的（显示"Create Root Node"），视口中只有一个网格地面和方向光。这就是你的起点——一个空荡荡的3D世界，等待你往里面添加内容。

步骤4：设置项目配置

点击菜单栏的Project Settings，这里有几个必须修改的配置。在General标签页下：Application Name 改为"代号：传说"；Application Icon 选一张图标图片（暂时跳过也行）；Application Run Path 暂时留空（后面创建主场景后再设置）。在Input Map标签页下，我们稍后统一添加按键映射。

配置文件的真相：你在这里修改的所有设置，最终都保存在项目根目录的project.godot文件中。这个文件是一个文本格式的配置文件，可以直接用文本编辑器打开修改。Git版本控制时这个文件必须提交，因为其他开发者需要相同的配置才能正常运行项目。

四、2.5D技术方案——为什么这么做

在开始搭建场景前，你必须理解2.5D的技术方案为什么是这样设计的。不理解原理，后面每一步都会觉得"为什么要这样做"。

4.1 什么是2.5D——三种方案对比

2.5D不是一种固定技术，而是一个概念——"看起来像3D但本质上是2D"或"3D场景里有2D角色"。在Godot中有三种实现2.5D的方案，各有优劣：

方案	场景层	角色层	优点	缺点
方案A	2D(TileMap)	2D(Sprite2D)+伪高度	最简单	Z轴是假的，无法做真正的高低差战斗
方案B	3D(TileMap3D)	2D(Sprite3D)	真Z轴+像素角色	Sprite3D性能开销稍大
方案C	3D(全部)	3D(模型)	最强大	失去了像素风格，美术成本暴增

表4-1 三种2.5D方案对比

我们选择方案B，原因如下：设计文档明确要求"3D场景+2D像素角色+真实Z轴"——方案A没有真Z轴，方案C没有像素风格，只有方案B同时满足三个需求。Boss的多阶段设计中明确有"跳跃到高处平台再俯冲"的机制，这需要真实的3D物理来处理角色在Z轴上的运动和碰撞，2D伪高度根本做不到。

4.2 Sprite3D——2D图片在3D世界中的桥梁

Sprite3D是方案B的核心节点。它的工作原理是：在3D空间中放置一个"广告牌"（Billboard）——一块永远面向摄像机的平面，上面贴着2D像素图。你可以把它理解为一块立在3D世界里的纸板人，它看起来是2D的，但存在于3D空间中，可以被3D光照照亮，可以和3D碰撞体交互。

Sprite3D有两个关键属性：texture（图片资源）和billboard（朝向模式）。billboard设为"Y-Billboard"意味着角色只在水平方向上旋转面向摄像机，但不会因为摄像机上下移动而倾斜——这是2.5D游戏的标准做法，你见过的大部分2.5D游戏角色都是这样渲染的。

原理：Sprite3D在底层创建了一个带纹理的四边形（Quad）mesh，每帧根据摄像机位置计算旋转角度，使四边形始终面向摄像机。渲染时使用Alpha Cut模式处理像素图中的透明区域（像素角色的轮廓不是矩形）。billboard模式控制旋转轴——Y-Billboard只绕Y轴旋转，保留角色的上下朝向。

4.3 CharacterBody3D——为什么不用2D物理

这是新手最容易困惑的问题：既然角色是2D像素，为什么用3D物理体（CharacterBody3D）而不是2D物理体（CharacterBody2D）？答案是：因为场景是3D的。CharacterBody2D只能在XY平面上移动，完全无法处理Z轴碰撞——当Boss跳到高处平台时，2D物理的角色根本无法判断"我站在平台上"还是"我在空中"，因为2D物理根本不知道什么是"高度"。

CharacterBody3D在3D空间中移动，使用move_and_slide()方法处理与3D碰撞体的交互。当角色跳到一个平台上时，引擎知道"角色脚下的碰撞体是地面"，角色会稳定地站在上面；当角色从平台上走下来时，引擎知道"脚下没有碰撞体了"，角色会开始下落。这一切都是3D物理引擎自动处理的，你不需要自己写任何碰撞检测代码。

原理：move_and_slide()的内部实现是：先计算当前帧的速度向量(velocity)，然后调用PhysicsServer3D的body_test_move()方法测试移动后的位置是否与任何碰撞体重叠。如果重叠，就计算滑动向量（沿碰撞面滑动），然后再次测试，直到找到不重叠的位置或达到最大滑动次数。这个过程自动处理了"站在地面上"、"撞墙停住"、"沿斜面滑下"等所有情况。

五、手把手：搭建2.5D测试场景

现在开始真正的操作。我会把每一步都写出来，包括在哪里点击、输入什么、为什么这么做。

5.1 创建3D根节点

步骤1：点击场景树面板左上角的"+"按钮（或按Ctrl+A），在弹出的节点列表中选择"Node3D"，点击"Create"。

为什么：Node3D是最基础的3D节点——它本身什么都不做，只是提供了一个3D坐标系（位置/旋转/缩放）。所有的3D节点都继承自Node3D，所以它是一切3D内容的根节点。就像HTML中的标签，本身不显示内容，但所有可见内容都放在它里面。

步骤2：选中刚创建的Node3D节点，按F2重命名为"TestArena"（测试竞技场）。

命名规范很重要！不然后面节点多了你根本分不清谁是谁。根节点用大驼峰命名法（PascalCase），比如TestArena、Player、BossGuard。子节点用下划线命名法（snake_case），比如sprite_3d、collision_shape。

步骤3：按Ctrl+S保存场景，文件名为"test_arena.tscn"，保存到项目的scenes/world/目录下（先创建这个文件夹）。

5.2 添加3D地面

步骤1：选中TestArena根节点，点"+"添加子节点，搜索"StaticBody3D"，创建它。重命名为"Ground"。

为什么用StaticBody3D：地面是不动的物体（Static = 静态），CharacterBody3D的move_and_slide()需要与StaticBody3D碰撞才能"站在地面上"。如果地面不是物理体，角色会直接穿过去掉入虚空。

步骤2：选中Ground节点，点"+"给它添加两个子节点：CollisionShape3D 和 MeshInstance3D。

为什么需要两个子节点：CollisionShape3D定义"碰撞的形状"（物理引擎用），MeshInstance3D定义"看到的樣子"（渲染用）。物理和视觉是完全独立的两套系统——你可以在视觉上把地面做得很复杂，但碰撞形状只需要一个简单的平面就够了。

步骤3：选中CollisionShape3D节点，在右侧属性面板中找到Shape属性，点击选择"New BoxShape3D"。然后在属性面板中设置Size为(30, 0.2, 20)——一个30米宽、0.2米厚、20米深的平台。

步骤4：选中MeshInstance3D节点，在属性面板中找到Mesh属性，点击选择"New BoxMesh"。设置Size与碰撞体相同(30, 0.2, 20)。在Material Overrides -> Material中创建一个新材质，设置Albedo Color为浅灰色(0.6, 0.6, 0.6)，这样地面的颜色和背景有区分。

步骤5：在视口中按鼠标中键拖拽可以旋转3D视角，滚轮缩放，Shift+中键平移。调整到一个能看清地面的斜45度视角——这就是2.5D游戏的典型观察角度。

5.3 添加3D摄像机

步骤1：选中TestArena根节点，添加子节点"Camera3D"，重命名为"MainCamera"。

步骤2：在属性面板中设置Position为(0, 12, 12)——摄像机在地面中心上方12米、后方12米的位置。设置Rotation为(-45, 0, 0)——摄像机向下俯视45度。

为什么是这个角度：俯视45度是2.5D游戏的经典视角（如暗黑破坏神、八方旅人），玩家既能看到角色的正面，又能感受到场景的纵深。Position和Rotation的数值可能需要反复调整，目标是"角色在画面中下部，上方留出足够空间显示场景"。

步骤3：在属性面板中找到Projection，设为"Perspective"（透视投影）。FOV设为50（比默认70更窄，减少透视变形，让2D像素角色看起来不扭曲）。

FOV是什么：FOV（Field of View）是摄像机的视野角度。FOV越大，透视变形越严重——近处的物体变得很大，远处的变得很小，2D像素角色在这种镜头下会严重扭曲。2.5D游戏通常用40-55的FOV，比FPS游戏的70-90小得多，这样透视变形可控，角色看起来更"平面化"。

步骤4：点击视口右上角的"Preview"按钮（或选中Camera3D后按Ctrl+P），预览摄像机看到的画面。你应该能看到灰色地面的俯视效果。

5.4 添加3D光照

步骤1：选中TestArena根节点，添加子节点"DirectionalLight3D"，重命名为"SunLight"。

步骤2：设置Rotation为(-60, 30, 0)——模拟从左上方斜射的阳光。设置Light Color为暖白色(1.0, 0.95, 0.85)。设置Shadow Enabled为true——开启阴影，这对2.5D游戏的立体感至关重要。

为什么需要光照：没有光照的3D场景看起来是"死"的——所有表面都是同一个颜色，没有任何立体感。光照让朝向光源的面变亮、背向的面变暗，自然产生了明暗差异，让你能感知到"这个方块是3D的"。阴影进一步强化了立体感——角色的影子投射在地面上，让你知道"角色站在地面上方"。2D像素角色在3D光照下会产生非常有趣的视觉效果：像素图本身不受光照影响（因为它是一张贴图），但Sprite3D的材质可以被3D光照照亮，产生微妙的明暗变化。

5.5 放置2D像素角色（Sprite3D）

步骤1：选中TestArena根节点，添加子节点"CharacterBody3D"，重命名为"Player"。这是我们角色的物理体根节点，负责3D移动和碰撞。

步骤2：选中Player节点，给它添加三个子节点：Sprite3D、CollisionShape3D、AnimationPlayer。分别重命名为sprite、collision、animator。

为什么是这个结构：CharacterBody3D负责物理移动，Sprite3D负责视觉显示，CollisionShape3D负责碰撞检测，AnimationPlayer负责播放动画。这四个职责完全独立，分别由不同节点承担——这就是Godot"组合优于继承"的设计哲学。

步骤3：选中sprite(Sprite3D)节点，在属性面板中找到Texture属性。现在你还没有像素图资源，先不用填——后面我会教你怎么制作和导入像素图。找到Billboard属性，设为"Y-Billboard"——让角色始终水平面向摄像机。找到Texture Filter属性，设为"Nearest"——关闭纹理过滤，保持像素锐利。

Nearest为什么重要：默认的Linear过滤会在像素之间做插值，让像素图的边缘变得模糊——这对照片是好的，但对像素风是灾难。Nearest模式不做插值，每个像素都是清晰的方块，这才是像素风游戏的正确渲染方式。

步骤4：选中collision(CollisionShape3D)节点，在属性面板中Shape属性选择"New CapsuleShape3D"（胶囊形状，接近人形角色的碰撞体）。设置Radius为0.4、Height为1.6——大约是一个人的碰撞范围。

步骤5：选中Player(CharacterBody3D)节点，设置Position为(0, 1, 0)——将角色放在地面上方1米处。为什么是1米？因为胶囊碰撞体的高度是1.6米，它的中心点在0.8米处，所以需要把Position.y设为0.8以上角色才不会半截陷入地面。设为1是为了保险，运行时物理引擎会自动把角色"放"在地面上。

5.6 添加角色移动脚本

步骤1：选中Player(CharacterBody3D)节点，点击场景树面板右上角的"附加脚本"图标（滚动条图标），或右键选择"Attach Script"。在弹出的对话框中：语言选"GDScrip"，路径设为"scripts/player_move.gd"，模板选"Basic Movement"（Godot 4.6.2的CharacterBody3D移动模板）。点击创建。

步骤2：Godot会自动生成一段基础移动代码。你会看到编辑器切换到了脚本编辑模式，里面有一段约30行的GDScript代码。这段代码已经包含了move_and_slide()的调用，但默认是第一人称控制器的参数，我们需要改成2.5D动作游戏的参数。

步骤3：找到代码中的SPEED常量，将值从5.0改为8.0（动作RPG的移动速度应该更快一些）。找到JUMP_VELOCITY，保持4.5不变。找到gravity变量，确认它从Project Settings获取（代码里默认就是这样的）。

步骤4：这段默认代码使用WASD移动，但它是"相对摄像机方向"移动的——按W角色会向摄像机面向的方向走，而不是向世界坐标的-Z方向。这对于2.5D俯视角游戏来说正好合适，不需要修改移动逻辑。

步骤5：按F6运行当前场景（test_arena.tscn），你应该能：

- 1. 用WASD移动角色（虽然角色还看不到因为没有图片，但可以在输出面板看到position变化）
- 2. 按空格键跳跃（同样看不到，但可以观察到Position.y值的变化）
- 3. 角色不会穿过地面（物理碰撞生效）

如果运行后报错，检查：1) 场景是否保存了 2) Player节点的脚本路径是否正确 3) Ground的CollisionShape3D是否有Shape（不是empty）。这三个是最常见的新手问题。

六、设置按键映射——InputMap

目前WASD和空格是Godot默认的InputMap预设，可以正常工作。但我们的游戏需要更多按键——攻击(J)、闪避(K)、技能1-4(1/2/3/4键)等。这些都需要在InputMap中定义。

步骤1：打开Project -> Project Settings，切换到Input Map标签页。

步骤2：在顶部的输入框中输入动作名称，点击"Add"添加。依次添加以下动作：

动作名称	按键1	按键2	用途
move_up	W	Up Arrow	向上移动
move_down	S	Down Arrow	向下移动
move_left	A	Left Arrow	向左移动
move_right	D	Right Arrow	向右移动
jump	Space	(无)	跳跃
attack	J	Left Mouse	普攻/重击
dodge	K	Shift	闪避
skill_1	1	(无)	技能1

动作名称	按键1	按键2	用途
skill_2	2	(无)	技能2
skill_3	3	(无)	技能3
skill_4	4	(无)	技能4/终极
interact	E	(无)	与NPC/物品交互

表6-1 按键映射配置

步骤3：每个动作添加按键的方法——点击动作名右方的"+"按钮，在弹出的对话框中按下你想绑定的键（比如W），然后点击OK。同一个动作可以绑定多个键（比如move_up绑定W和上箭头）。

为什么不直接在代码里用Input.is_key_pressed(KEY_J)：因为InputMap让你在不改代码的情况下更换按键。玩家想用方向键移动？只需在设置界面改一下，代码完全不用动。这就是"数据驱动设计"——把"做什么"和"怎么触发"分开。

七、像素图资源——制作与导入

7.1 像素图从哪里来

在项目早期，你不需要精美的像素图。有三种快速获取临时素材的方式：

方式一：在Aseprite（像素画软件，\$20买断）中画一个32x32或48x48的简单人形。不需要精细——一个方块头+长方形身体+两条腿的"火柴人"级别就够了。每个方向画4帧走路动画（正面、背面、左、右），总共16帧。导出为水平spritesheet（一行排列）。

方式二：从OpenGameArt.org或itch.io搜索"pixel character sprite"，下载免费素材。注意查看许可证（License），CC0或CC-BY是最安全的。

方式三：暂时用纯色方块代替。在Sprite3D的Texture属性中不填图片，而是给它添加一个CanvasItem材质，设置一个纯色。这虽然难看但功能完整——你可以先验证移动和碰撞，后面再替换真正的像素图。

7.2 导入设置——最容易被忽略的步骤

当你把PNG文件放入项目文件夹后，Godot会自动导入它。但默认的导入设置对像素图是错误的——必须修改！

步骤1：在文件系统面板中找到你的PNG文件，选中它。

步骤2：右侧会出现"Import"标签页（注意不是Inspector标签页）。找到"Texture"类型的导入选项，将Filter从"Linear"改为"Nearest"。这个设置控制图片缩放时的过滤方式——Nearest保持像素锐利，Linear会让像素变模糊。

步骤3：点击"Reimport"按钮重新导入。现在你的像素图在场景中显示时就是锐利的了。

批量设置：选中所有像素图PNG文件，一起改Filter为Nearest后Reimport。这个设置是按文件保存的，每个新加入的PNG都需要手动改。如果你觉得麻烦，可以在project.godot中添加一行全局默认设置。

八、移动脚本逐行解读

现在你已经有了移动脚本（player_move.gd），但可能看不懂每行代码的意思。下面逐行解释，确保你理解每一行在做什么、为什么这么做。

```
extends CharacterBody3D
```

这行告诉Godot：这个脚本是CharacterBody3D的扩展。扩展（extends）意味着这个脚本"继承"了CharacterBody3D的所有功能（move_and_slide、velocity等），并可以添加自己的逻辑。你可以理解为"在这个预制零件上追加自定义功能"。

```
const SPEED = 8.0
const JUMP_VELOCITY = 4.5
```

const声明常量——值不会在运行时改变。SPEED是移动速度（8米/秒），JUMP_VELOCITY是跳跃初速度（4.5米/秒）。把数值写在const里而不是直接写在代码中（"魔法数字"），方便后续调整——改一个数字就能影响整个移动系统。

```
var gravity = ProjectSettings.get_setting("physics/3d/default_gravity")
```

从项目设置中读取3D重力加速度（默认9.8）。使用ProjectSettings而不是硬编码数字，这样如果你在项目设置中修改了重力值，脚本会自动使用新值。

```
func _physics_process(delta):
```

这是Godot的内置回调函数，每帧调用一次（物理帧，默认60次/秒）。delta参数是上一帧到这一帧的时间间隔（约0.0167秒）。所有物理相关的逻辑（移动、碰撞、重力）都必须放在_physics_process中，而不是_process中——_process是渲染帧，帧率不稳定，不适合物理计算。

```
if not is_on_floor():
    velocity.y -= gravity * delta
```

is_on_floor()是CharacterBody3D的内置方法，检测角色是否站在地面上。如果不在地面上（在空中），就对velocity.y施加向下的重力加速度。每帧velocity.y减少gravity*delta（约0.163），角色下落速度越来越快，模拟真实的重力加速效果。

```
if Input.is_action_just_pressed("jump") and is_on_floor():
    velocity.y = JUMP_VELOCITY
```

is_action_just_pressed()检测按键是否"刚刚被按下"（只触发一次，不是持续按住）。只有在地面上的角色才能跳跃——如果没有is_on_floor()检查，玩家可以在空中无限跳（"空中跳跃"bug）。设置velocity.y为正值（4.5），角色向上运动，然后重力会逐渐把它拉回来。

```
var input_dir = Input.get_vector("move_left", "move_right", "move_up", "move_down")
var direction = (transform.basis * Vector3(input_dir.x, 0, input_dir.y)).normalized()
if direction:
    velocity.x = direction.x * SPEED
    velocity.z = direction.z * SPEED
else:
    velocity.x = move_toward(velocity.x, 0, SPEED)
    velocity.z = move_toward(velocity.z, 0, SPEED)
```

这段代码做三件事：第一，Input.get_vector()将四个方向的按键输入转换为一个2D向量（x=左右，y=上下），值在-1到1之间。第二，将2D输入向量转换为3D世界坐标方向——transform.basis是角色当前的朝向矩阵，乘以输入向量后得到"角色朝向的左右前后"，这样按W就是"向前走"而不是"向世界坐标-Z走"。normalized()确保对角线移动不会比直线移动快（否则斜向走会快1.414倍）。第三，如果有输入方向就设置velocity为目标速度，没有输入就用move_toward()逐渐减速到0（产生惯性滑行效果）。

```
move_and_slide()
```

这是CharacterBody3D最重要的方法。它根据当前velocity移动角色，并自动处理与碰撞体的交互——碰到墙壁就停住，站在地面上就不会穿过去，遇到斜面会沿斜面滑动。你不需要自己写任何碰撞检测代码，move_and_slide()内部已经处理了一切。

九、新手常见问题与排错

问题	原因	解决方法
角色穿过了地面	Ground没有CollisionShape3D，或Shape为空	给Ground添加CollisionShape3D，设置BoxShape3D
角色半截陷入地面	Position.y太低，碰撞体中心在地面以下	把Player的Position.y提高（至少0.8）
角色一直往下掉	Ground是MeshInstance3D不是StaticBody3D	地面必须用StaticBody3D作为根节点
看不到角色	Sprite3D没有Texture	先给Texture填一张图片，或用纯色材质代替
角色图片很模糊	像素图导入时没改Filter为Nearest	选中PNG -> Import -> Filter改为Nearest -> Reimport
按空格没反应	InputMap中没有"jump"动作，或没绑定Space键	Project Settings -> Input Map -> 添加jump动作绑定Space
运行时报错"Scrip t not found"	脚本路径不对或场景没保存	检查脚本文件是否存在，重新保存场景(Ctrl+S)
场景一运行就黑屏	没有Camera3D，或Camera3D没有激活	确保场景中有Camera3D且Current属性勾选
角色向左走但图片朝右	Sprite3D的Flip H属性需要动态设置	在脚本中根据velocity.x的正负设置flip_h
3D场景中没有阴影	DirectionalLight3D的Shadow没有开启	选中灯光 -> 勾选Shadow Enabled

表9-1 新手常见问题排查表

十、完成基础场景后接下来做什么

当你成功搭建了2.5D测试场景、角色能移动和跳跃后，你就完成了里程碑1

Sprint

1.1的核心目标。接下来的学习路线按优先级排列：

顺序	学习内容	你会获得什么	对应路线图位置
1	AnimationPlayer + Sprite3D动画	角色有走路/攻击/待机动画	Sprint 1.2
2	Area3D碰撞检测 + 伤害系统	攻击能打中敌人造成伤害	Sprint 1.2
3	状态机(State Machine)模式	角色在待机/走路/攻击/闪避间切换	Sprint 1.2
4	敌人AI基础（巡逻+追击）	第一个小怪能在场景中巡逻和追击玩家	Sprint 1.3
5	Boss多阶段设计	Boss有3个阶段，每阶段行为不同	Sprint 1.3

顺序	学习内容	你会获得什么	对应路线图位置
6	Resource自定义数据	怒气系统/技能数据/装备数据	Sprint 2.1
7	UI系统(Control节点)	血条/怒气条/技能栏/背包	Sprint 2.2

表10-1 后续学习路线

每个内容我都会给你写一份同样详细的教学文档——先讲原理，再讲操作步骤，最后讲常见问题。你只需要告诉我"我完成了基础场景，准备学下一个"，我就会生成对应的教学。

最重要的一点：不要跳步。如果你还没成功让角色在地面上走起来，就不要急着做攻击动画。Godot是"增量式开发"——每一步都是下一步的基础，跳过任何一步都会在后面花10倍的时间回来补。